

Big Data Analytics and Visualization Using AWS and PySpark

Noopur Shekhar Divekar

Objective

The primary objective of this project was to design and implement a scalable, end-to-end Big Data analytics pipeline using Amazon Web Services (AWS) and PySpark. The project aimed to process large-scale datasets efficiently in a distributed cloud environment to derive actionable business insights. Furthermore, it incorporates automated machine learning via AWS SageMaker to predict customer churn and utilizes Power BI to visualize key metrics, ensuring a comprehensive analysis from raw data to business insights.

Project Workflow

1. Dataset Selection

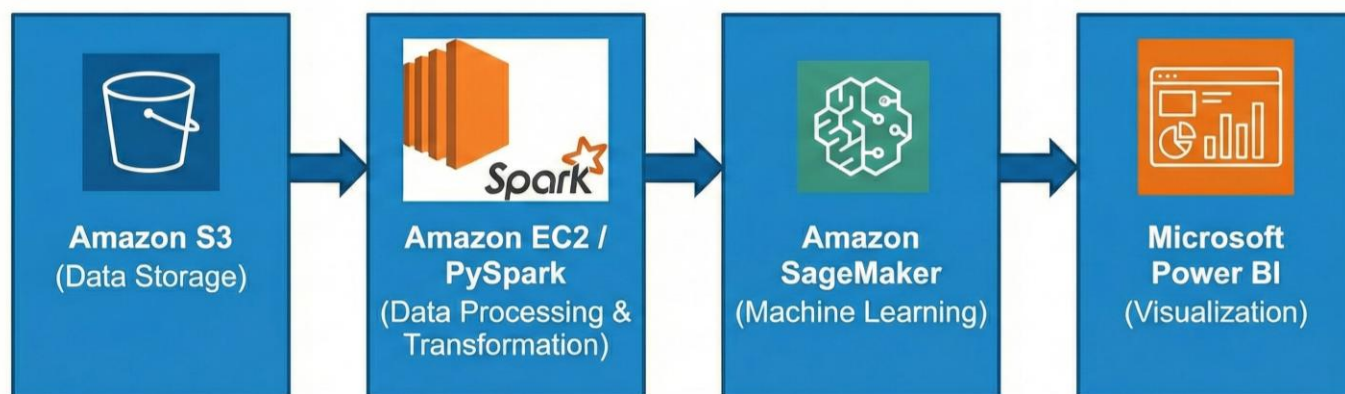
The **Churn Modelling** dataset contains 10,000 records of bank customers, designed for analyzing customer attrition. It consists of 14 columns capturing demographic, financial, and behavioral attributes.

Unique identifiers include CustomerId and Surname, while core demographic data covers Geography, Gender, and Age. Financial health is represented by CreditScore, Balance, and EstimatedSalary, alongside behavioral metrics such as Tenure, NumOfProducts, HasCrCard, and IsActiveMember.

The target variable, Exited, is binary (1 for churned, 0 for retained), making the dataset ideal for classification tasks like churn prediction. The combination of numerical and categorical data supports robust predictive modeling and customer segmentation, enabling banks to identify at-risk clients and develop data-driven retention strategies.

RowNum	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfPr	HasCrCar	IsActiveM	Estimated	Exited
1	15634602	Hargrave	619	France	Female	42	2	0	1	1	1	101348.9	1
2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	112542.6	0
3	15619304	Onio	502	France	Female	42	8	159660.8	3	1	0	113931.6	1
4	15701354	Boni	699	France	Female	39	1	0	2	0	0	93826.63	0
5	15737888	Mitchell	850	Spain	Female	43	2	125510.8	1	1	1	79084.1	0
6	15574012	Chu	645	Spain	Male	44	8	113755.8	2	1	0	149756.7	1
7	15592531	Bartlett	822	France	Male	50	7	0	2	1	1	10062.8	0
8	15656148	Obinna	376	Germany	Female	29	4	115046.7	4	1	0	119346.9	1
9	15792365	He	501	France	Male	44	4	142051.1	2	0	1	74940.5	0
10	15592389	H?	684	France	Male	27	2	134603.9	1	1	1	71725.73	0
11	15767821	Bearce	528	France	Male	31	6	102016.7	2	0	0	80181.12	0
12	15737173	Andrews	497	Spain	Male	24	3	0	2	1	0	76390.01	0
13	15632264	Kay	476	France	Female	34	10	0	2	1	0	26260.98	0
14	15691483	Chin	549	France	Female	25	5	0	2	0	0	190857.8	0
15	15600882	Scott	635	Spain	Female	35	7	0	2	1	1	65951.65	0
16	15643966	Goforth	616	Germany	Male	45	3	143129.4	2	0	1	64327.26	0
17	15737452	Romeo	653	Germany	Male	58	1	132602.9	1	1	0	5097.67	1
18	15788218	Henderso	549	Spain	Female	24	9	0	2	1	1	14406.41	0
19	15661507	Muldrow	587	Spain	Male	45	6	0	1	0	0	158684.8	0
20	15568982	Hao	726	France	Female	24	6	0	2	1	1	54724.03	0

Pipeline Workflow



Architecture & Data Ingestion

2. Environment Setup

1. AWS S3 for Data Storage

- a. **Step 1:** Create an S3 bucket to store both raw and processed data.

The screenshot shows the AWS S3 console interface for 'General purpose buckets'. At the top, there are tabs for 'General purpose buckets' (selected) and 'Directory buckets'. Below the tabs, there's a section titled 'General purpose buckets (2)' with an 'Info' link. It includes buttons for 'Copy ARN', 'Empty', 'Delete', and 'Create bucket'. A search bar is present with the placeholder text 'Find buckets by name'. Below the search bar is a table listing two buckets:

Name	AWS Region	Creation date
ndivekar-processed-data	US East (Ohio) us-east-2	December 4, 2025, 13:09:23 (UTC-05:00)
ndivekar-raw-data	US East (Ohio) us-east-2	December 3, 2025, 11:19:13 (UTC-05:00)

- b. **Step 2:** Upload the raw dataset to the S3 bucket.

The screenshot shows the AWS S3 console interface for the 'ndivekar-raw-data' bucket. At the top, there are tabs for 'Objects' (selected), 'Metadata', 'Properties', 'Permissions', 'Metrics', 'Management', and 'Access Points'. Below the tabs, there's a section titled 'Objects (1)' with an 'Info' link. It includes buttons for 'Copy S3 URI', 'Copy URL', 'Download', 'Open t...', 'Delete', 'Actions', 'Create folder', and 'Upload'. A search bar is present with the placeholder text 'Find objects by prefix'. Below the search bar is a table listing one object:

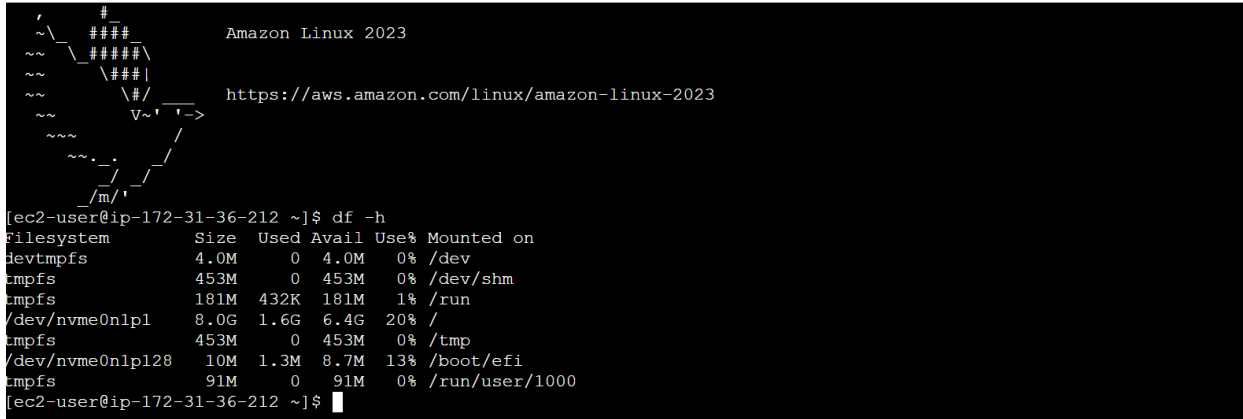
Name	Type	Last modified	Size	Storage class
Churn_Modelling.csv	csv	December 4, 2025, 12:58:18 (UTC-05:00)	668.8 KB	Standard

2. Linux Environment with PySpark

- a. **Step 1:** Set up a Linux-based environment using an AWS EC2 instance.

The screenshot shows the AWS Management Console interface for EC2 instances. At the top, there are tabs for 'Instances (1)' (selected) and 'Info'. It includes buttons for 'Connect', 'Instance state', 'Actions', and 'Launch instances'. A search bar is present with the placeholder text 'Find Instance by attribute or tag (case-sensitive)'. Below the search bar is a table listing one instance:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status
bda-miniproject	i-05bb765dedc533cc0	Running	t3.micro	Initializing	View alarms +



5. Set the Environment Variables: The SPARK_HOME environment variable was set to

the directory where the Spark is extracted. This allows Spark to be accessed from anywhere in the terminal. Do the following changes in the `~/.bashrc`:

```
export SPARK_HOME=/opt/spark
```

```
export PATH=$SPARK_HOME/bin:$PATH
```

```
export JAVA_HOME=/usr/lib/jvm/java-1.8.0-amazon-corretto.x86_64/jre
```

```
export PATH=$JAVA_HOME/bin:$PATH
```

6. Install Spark using pip3:

[illegible]

c. Step 3: Configure AWS CLI to interact with S3 buckets.

Successfully installed and configured the AWS CLI on EC2 instance to connect with the S3 buckets. To manage and access S3 buckets, such as ndivekar-raw-data, directly from the command line on EC2 instance.

```
[ec2-user@ip-172-31-41-39 ~]$ sudo yum install aws-cli -y
Last metadata expiration check: 0:42:21 ago on Thu Dec 4 15:21:12 2025.
Package awscli-2-2.30.4-1.amzn2023.0.1.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-41-39 ~]$ aws --version
aws-cli/2.30.4 Python/3.9.24 Linux/6.1.158-178.288.amzn2023.x86_64 source/x86_64.amzn.2023
[ec2-user@ip-172-31-41-39 ~]$
```

The image below confirms that the connection between the EC2 instance and S3 has been successfully established. It displays a list of S3 buckets, verifying that the instance can interact with S3 using the provided credentials and configurations. This indicates that the setup for accessing S3 from the EC2 instance, whether via the AWS CLI or Boto3, is

functioning as expected.

```
[ec2-user@ip-172-31-41-39 ~]$ aws s3 ls
2025-12-04 18:09:23 ndivekar-processed-data
2025-12-03 16:19:14 ndivekar-raw-data
```

3. Data Pipeline Tasks

Task 1: Data Ingestion from S3

- **Objective:** Pull raw data from S3 into the PySpark environment.
- **Steps:**
 1. Use AWS CLI or PySpark's built-in S3 support to load the dataset directly.
 2. Confirm successful ingestion by inspecting the dataset.

This Python script explains how to retrieve a CSV file from an AWS S3 bucket and load it into PySpark for data processing. Boto3 connects to S3, downloads Churn_Modelling.csv from the ndivekar-raw-data bucket, and saves it locally. A PySpark session is set up for distributed data processing. The script then reads the local CSV file into a PySpark Data Frame, which includes headers and automatically infers structure. Finally, the schema for the dataset is printed to ensure its structure.

s3_data_ingestion.py

```

import boto3
from pyspark.sql import SparkSession

# Initialize S3 client
s3_client = boto3.client('s3')

# S3 Bucket and File Details
bucket_name = "ndivekar-raw-data"
file_key = "Churn_Modelling.csv"
local_file_path = "Churn_Modelling.csv"

# Step 1: Download the dataset from S3
try:
    print(f"Downloading {file_key} from bucket {bucket_name}...")
    s3_client.download_file(bucket_name, file_key, local_file_path)
    print(f"File downloaded to {local_file_path}")
except Exception as e:
    print(f"Error downloading file: {e}")
    exit(1)

# Step 2: Initialize PySpark Session
spark = SparkSession.builder \
    .appName("S3DataIngestion") \
    .getOrCreate()

# Step 3: Load the Dataset into PySpark
try:
    print(f"Loading dataset into PySpark from {local_file_path}...")
    df = spark.read.csv(local_file_path, header=True, inferSchema=True)
    print("Dataset loaded successfully!")

    # Step 4: Inspect the Dataset
    print("Schema of the dataset:")
    df.printSchema()

    print("Sample rows from the dataset:")
    df.show(5)
except Exception as e:
    print(f"Error loading dataset into PySpark: {e}")
Finally:
    spark.stop()

```

```
ec2-user@ip-172-31-41-39 ~]$ ls -l
total 1072876
-rw-r--r--. 1 ec2-user ec2-user 684858 Dec 4 19:07 Churn_Modelling.csv
-rw-r--r--. 1 ec2-user ec2-user 3626 Dec 4 19:07 s3_data_ingestion.py
drwxr-xr-x. 14 ec2-user ec2-user 16384 Sep 2 03:39 spark-4.0.1-bin-hadoop3
-rw-r--r--. 1 ec2-user ec2-user 548955321 Sep 2 04:08 spark-4.0.1-bin-hadoop3.tgz
-rw-r--r--. 1 ec2-user ec2-user 548955321 Sep 2 04:08 spark-4.0.1-bin-hadoop3.tgz.1
```

The image below shows the output from the `s3_data_ingestion.py` file. It outputs the schema of the dataset and it also displays the first 5 rows of the dataset.

```
25/12/04 19:23:26 WARN NativeCodeLoader: Unable to load native-heapooc library for your platform... using builtin-java classes where applicable
Loading dataset into PySpark from Churn_Modelling.csv...
Dataset loaded successfully!
Schema of the dataset:
root
 |-- RowNumber: integer (nullable = true)
 |-- CustomerId: integer (nullable = true)
 |-- Surname: string (nullable = true)
 |-- CreditScore: integer (nullable = true)
 |-- Geography: string (nullable = true)
 |-- Gender: string (nullable = true)
 |-- Age: integer (nullable = true)
 |-- Tenure: integer (nullable = true)
 |-- Balance: double (nullable = true)
 |-- NumOfProducts: integer (nullable = true)
 |-- HasCrCard: integer (nullable = true)
 |-- IsActiveMember: integer (nullable = true)
 |-- EstimatedSalary: double (nullable = true)
 |-- Exited: integer (nullable = true)

Sample rows from the dataset:
RowNumber|CustomerId| Surname|CreditScore|Geography|Gender|Age|Tenure| Balance|NumOfProducts|HasCrCard|IsActiveMember|EstimatedSalary|Exited
-----
1| 15634602|Hargrave| 619| France|Female| 42| 2| 0.0| 1| 1| 101348.88| 1
2| 15647311| Hill| 608| Spain|Female| 41| 1| 83807.86| 1| 0| 1| 112542.58| 0
3| 15619304| Onio| 502| France|Female| 42| 8| 159660.8| 3| 1| 1| 113931.57| 1
4| 15701354| Boni| 699| France|Female| 39| 1| 0.0| 2| 0| 0| 93826.63| 0
5| 15737888| Mitchell| 850| Spain|Female| 43| 2| 125510.82| 1| 1| 1| 79084.1| 0

only showing top 5 rows
```

Checking for missing values:

This PySpark software detects missing values in the dataset. It begins by initializing a Spark Session and loading a CSV file into a PySpark DataFrame using schema inference. After showing the schema, it discovers missing values column by column by producing a binary column for each field, where null values are represented as 1 and non-values as 0. This script combines these binary columns using the sum function to compute the total count of null values in each column.

```

GNU nano 8.3                                     check_missing.py
import boto3
from pyspark.sql import SparkSession

# Initialize S3 client
s3_client = boto3.client('s3')

# S3 Bucket and File Details
bucket_name = "ndivekar-raw-data"
file_key = "Churn_Modelling.csv"
local_file_path = "Churn_Modelling.csv"

# Step 1: Download the dataset from S3
try:
    print(f"Downloading {file_key} from bucket {bucket_name}...")
    s3_client.download_file(bucket_name, file_key, local_file_path)
    print(f"File downloaded to {local_file_path}")
except Exception as e:
    print(f"Error downloading file: {e}")
    exit(1)

# Step 2: Initialize PySpark Session
spark = SparkSession.builder \
    .appName("S3DataIngestion") \
    .getOrCreate()

# Step 3: Load the Dataset into PySpark
try:
    print(f"Loading dataset into PySpark from {local_file_path}...")
    df = spark.read.csv(local_file_path, header=True, inferSchema=True)
    print("Dataset loaded successfully!")

    # Step 4: Inspect the Dataset
    print("Schema of the dataset:")
    df.printSchema()

    print("Sample rows from the dataset:")
    df.show(5)
except Exception as e:
    print(f"Error loading dataset into PySpark: {e}")
finally:
    spark.stop()

[ec2-user@ip-172-31-41-39 ~]$ nano check_missing.py
[ec2-user@ip-172-31-41-39 ~]$ python3 check_missing.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 19:34:48 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Schema of the dataset:
root
 |-- RowNumber: integer (nullable = true)
 |-- CustomerId: integer (nullable = true)
 |-- Surname: string (nullable = true)
 |-- CreditScore: integer (nullable = true)
 |-- Geography: string (nullable = true)
 |-- Gender: string (nullable = true)
 |-- Age: integer (nullable = true)
 |-- Tenure: integer (nullable = true)
 |-- Balance: double (nullable = true)
 |-- NumOfProducts: integer (nullable = true)
 |-- HasCrCard: integer (nullable = true)
 |-- IsActiveMember: integer (nullable = true)
 |-- EstimatedSalary: double (nullable = true)
 |-- Exited: integer (nullable = true)

Count of missing (null) values per column:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| RowNumber | CustomerId | Surname | CreditScore | Geography | Gender | Age | Tenure | Balance | NumOfProducts | HasCrCard | IsActiveMember | EstimatedSalary | Exited |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

```


1. **Data Transformation:** Create at least 2 new columns (e.g., **Year**, **Month**) to aid in analysis.

Encoding Categorical Features:

This is the `encode.py` file, which encodes columns (Geography and Gender) in a dataset with `StringIndexer`. It looks for existing encoded columns and eliminates them to avoid conflicts. The changed dataset is saved to a new CSV file, and a sample of the data is presented.

```

GNU nano 8.3
encoder.py
from pyspark.sql import SparkSession
from pyspark.ml.feature import StringIndexer

# Initialize SparkSession
spark = SparkSession.builder \
    .appName("CreateNewEncodedDataset") \
    .getOrCreate()

# Load the dataset
file_path = "Churn_Modelling.csv" # Update with the actual path
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Columns to encode
categorical_columns = ["Geography", "Gender"]

# Apply StringIndexer to each categorical column
for column in categorical_columns:
    output_column = f"{column}_Index"
    # Drop the column if it already exists
    if output_column in df.columns:
        df = df.drop(output_column)
    indexer = StringIndexer(inputCol=column, outputCol=output_column)
    df = indexer.fit(df).transform(df)

# Save the encoded dataset to a new CSV file
encoded_file_path = "Churn_Modelling_Encoded.csv" # Update the path as needed
df.write.csv(encoded_file_path, header=True, mode="overwrite")

# Show the resulting dataset
df.show(10)

# Stop SparkSession
spark.stop()

```

Output:

```
[ec2-user@ip-172-31-41-39 ~]$ python3 encoder.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
Do adjust logging level using sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 13:37:43 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited	Geography_Index	Gender_Index
1	15634602	Hargrave	619	France	Female	42	2	0.0	1	1	1	101348.88	1	0.0	1.0
2	15643101	Hill	608	Spain	Female	41	1	113807.86	1	0	1	112542.58	0	2.0	1.0
3	15619304	Onio	502	France	Female	42	8	159660.8	3	1	0	113931.57	1	0.0	1.0
4	15701344	Bonini	699	France	Female	39	1	0.0	1	1	1	93826.63	0	0.0	1.0
5	15737888	Mitchell	850	France	Female	43	2	125510.82	1	1	1	79084.1	0	2.0	1.0
6	15574012	Chui	645	Spain	Male	44	8	113755.78	2	1	0	149756.71	1	2.0	0.0
7	15592531	Bartlett	822	France	Male	50	7	0.0	2	1	1	10662.8	0	0.0	0.0
8	15656148	Obinna	376	Germany	Female	29	4	115046.74	4	1	0	119346.88	1	1.0	1.0
9	15792365	He	501	France	Male	44	4	142051.07	2	0	1	74940.5	0	0.0	0.0
10	1592389	H7	684	France	Male	27	2	134603.88	1	1	1	71725.73	0	0.0	0.0

```
only showing top 10 rows
```

Normalizing features:

This is the normalize.py code, which normalizes the Balance and EstimatedSalary columns with MinMaxScaler. It converts these columns into a feature vector, scales them from 0 to 1, and extracts the normalized values into a new column.

```
GNU nano 8.3 normalize.py
from pyspark.sql import SparkSession
from pyspark.ml.feature import MinMaxScaler
from pyspark.ml.feature import VectorAssembler
from pyspark.sql.functions import udf
from pyspark.ml.linalg import DenseVector
from pyspark.sql.types import DoubleType

# Initialize SparkSession
spark = SparkSession.builder \
    .appName("NormalizeColumns") \
    .getOrCreate()

# Load the dataset
file_path = "Churn Modelling Encoded.csv"
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Prepare columns to normalize
columns_to_normalize = ["Balance", "EstimatedSalary"]

# Create VectorAssembler for MinMaxScaler input
assembler = VectorAssembler(inputCols=columns_to_normalize, outputCol="features")
df_vectorized = assembler.transform(df)

# Apply MinMaxScaler
scaler = MinMaxScaler(inputCol="features", outputCol="scaledFeatures")
scaler_model = scaler.fit(df_vectorized)
df_scaled = scaler_model.transform(df_vectorized)

# Define UDFs to extract normalized values
def extract_first_element(vector):
    return float(vector[0])

def extract_second_element(vector):
    return float(vector[1])

udf_extract_first = udf(extract_first_element, DoubleType())
udf_extract_second = udf(extract_second_element, DoubleType())

# Create new normalized columns
df_normalized = df_scaled.withColumn("Normalized_Balance", udf_extract_first("scaledFeatures")) \
    .withColumn("Normalized_EstimatedSalary", udf_extract_second("scaledFeatures"))

# Drop the unsupported columns before saving
```

Output:

```
[ec2-user@ip-172-31-41-39 ~]$ python3 normalize.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 19:41:34 WARN NativeCodeLoader: Unable to load native-heapdump library for your platform... using builtin-java classes where applicable

+-----+
|RowNumber|CustomerId|Surname|CreditScore|Geography|Gender|Age|Tenure|Balance|NumOfProducts|HasCrCard|IsActiveMember|EstimatedSalary|Exited|Geography_Index|Gender_Index|Normalized_Balance|Normalized_EstimatedSalary|
+-----+
|1|15634602|Hargrave|619|France|Female|42|2|0.0|1|1|1|101348.88|1|0.0|1.0|0.0|0.506734893|
|2|15647311|Hill|608|Spain|Female|41|1|83807.86|1|0|1|112542.58|0|2.0|1.0|0.3340314786772591|0.562708738|
|3|15619304|Onio|502|France|Female|42|8|159640.8|3|1|0|113931.57|1|0.0|1.0|0.6363571759354565|0.569634351|
|4|15701354|Bonil|699|France|Female|39|1|0.0|2|0|0|93826.63|0|0.0|1.0|0.0|0.469120050|
|5|15737888|Mitchell|850|Spain|Female|43|2|125510.82|1|1|1|79084.1|0|2.0|1.0|0.5002462155052676|0.395400360|
|6|15574012|Chu|645|Spain|Male|44|8|113755.78|2|1|0|149756.71|1|2.0|0.0|0.45339436422174445|0.748797160|
|7|15592531|Bartlett|822|France|Male|50|7|0.0|2|1|1|10062.8|0|0.0|0.0|0.0|0.05026889951|
|8|15656148|Obinna|376|Germany|Female|29|4|115046.74|4|1|0|119346.88|1|1.0|1.0|0.4985397202505608|0.596733488|
|9|15792365|He|501|France|Male|44|4|142051.07|2|0|1|74940.5|0|0.0|0.0|0.5661703921301275|0.374680381|
|10|155923891|H7|684|France|Male|27|2|134603.88|1|1|1|71725.73|0|0.0|0.0|0.5364882610306042|0.3586049967|
|11|155923891|H7|684|France|Male|27|2|134603.88|1|1|1|71725.73|0|0.0|0.0|0.5364882610306042|0.3586049967|
```

2. Data Aggregation:

Total Balance by Geography:

```
GNU nano 8.3 totalbal_geo.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum as _sum

# Initialize SparkSession
spark = SparkSession.builder.appName("TotalBalanceByGeography").getOrCreate()

# Load the dataset
file_path = "Churn_Modelling.csv" # Update with the actual path
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Compute total balance by geography
total_balance_by_geography = df.groupBy("Geography").agg(_sum("Balance").alias("TotalBalance"))

# Show results
total_balance_by_geography.show()

# Stop SparkSession
spark.stop()
```

This PySpark code calculates the dataset's total bank balance, organized by geography. It creates a SparkSession, loads a CSV file, and conducts a group-by operation on the "geography" column before summing the "Balance" column. The results are displayed, and SparkSession is terminated following the execution.

Output:

```
[ec2-user@ip-172-31-41-39 ~]$ python3 totalbal_geo.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 19:48:23 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
+-----+
|Geography|      TotalBalance|
+-----+
|  Germany| 3.004028613799998E8|
|   France|3.1133247949000007E8|
|    Spain| 1.5312355201E8|
+-----+
```

Average Salary by Geography:

```
GNU nano 8.3 avgsalary_geo.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg

# Initialize SparkSession
spark = SparkSession.builder.appName("AverageSalaryByGeography").getOrCreate()

# Load the dataset
file_path = "Churn_Modelling.csv" # Update with the actual path
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Compute average salary by geography
average_salary_by_geography = df.groupBy("Geography").agg(avg("EstimatedSalary").alias("AverageSalary"))

# Show results
average_salary_by_geography.show()

# Stop SparkSession
spark.stop()
```

This PySpark code computes the average expected salary for each geography in the dataset. It creates a SparkSession, loads a CSV file, and does a group-by operation on the "Geography" column to calculate the average of the "EstimatedSalary" column. The results

are displayed, and the SparkSession terminates.

Output:

```
[ec2-user@ip-172-31-41-39 ~]$ python3 avgsalary_geo.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 19:49:09 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+
|Geography|   AverageSalary|
+-----+-----+
|Germany|101113.4351016342|
|France|99899.18081372172|
|Spain|99440.57228098497|
+-----+-----+
```

Average Balance by Tenure:

```
GNU nano 8.3                                                                    avgbal_tenure.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg

# Initialize SparkSession
spark = SparkSession.builder.appName("AverageBalanceByTenure").getOrCreate()

# Load the dataset
file_path = "Churn_Modelling.csv" # Update with the actual path
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Compute average balance by tenure
average_balance_by_tenure = df.groupBy("Tenure").agg(avg("Balance").alias("AverageBalance"))

# Show results
average_balance_by_tenure.show()

# Stop SparkSession
spark.stop()
```

This PySpark code calculates the average bank balance based on customer tenure from the dataset. It creates a SparkSession, loads a CSV file, and executes a group-by operation on the "Tenure" column to calculate the average of the "Balance" column. The results are displayed, and the SparkSession terminates.

Output:

```
[ec2-user@ip-172-31-41-39 ~]$ python3 avgbal_tenure.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 19:51:18 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+
|Tenure|   AverageBalance|
+-----+-----+
|1|78555.384222222226|
|6|74403.94263702171|
|3|77668.05988107044|
|5|76444.28085968386|
|9|76562.54401422766|
|4|74059.39406471176|
|8|77936.59825365855|
|7|75551.63442607006|
|10|73927.50400000003|
|2|76839.6264694657|
|0|79878.91062953991|
+-----+-----+
```

Top 10 Customers by Balance:

```
GNU nano 8.3 top10_bal.py

from pyspark.sql import SparkSession
from pyspark.sql.functions import desc

# Initialize SparkSession
spark = SparkSession.builder.appName("Top10CustomersByBalance").getOrCreate()

# Load the dataset
file_path = "Churn_Modelling.csv" # Update with the actual path
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Get top 10 customers by balance
top_10_customers_by_balance = df.select("CustomerId", "Surname", "Balance") \
    .orderBy(desc("Balance")) \
    .limit(10)

# Show results
top_10_customers_by_balance.show()

# Stop SparkSession
spark.stop()
```

This PySpark code determines the top ten customers with the highest bank balances. It launches a SparkSession, loads a CSV file, selects key columns ("CustomerId", "Surname", and "Balance"), and arranges the data in descending order by "Balance". The top ten results are shown, then the SparkSession terminates.

Output:

```
[ec2-user@ip-172-31-41-39 ~]$ python3 top10_bal.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 19:55:48 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
-----+-----+
[CustomerId| Surname| Balance|
-----+-----+
| 15757408| Lo|250898.09|
| 15715622| To Rot|238387.56|
| 15714241| Haddon|222267.63|
| 15571958| McIntosh| 221532.8|
| 15586674| Shaw|216109.88|
| 15599131| Dilke|214346.96|
| 15594408| Chia| 213146.2|
| 15769818| Moore| 212778.2|
| 15620268| Thomson|212696.32|
| 15780212| Mao|212692.97|
-----+-----+
```

Total Active Customers by Region:

```
GNU nano 8.3 actcust_reg.py

from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# Initialize SparkSession
spark = SparkSession.builder.appName("ActiveCustomersByRegion").getOrCreate()

# Load the dataset
file_path = "Churn_Modelling.csv" # Update with the actual path
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Count active customers by region
active_customers_by_region = df.filter(col("IsActiveMember") == 1) \
    .groupBy("Geography").count() \
    .withColumnRenamed("count", "ActiveCustomers")

# Show results
active_customers_by_region.show()

# Stop SparkSession
spark.stop()
```

This PySpark code counts the number of active customers grouped by "Geography". It

initiates a SparkSession, loads a CSV file, filters rows where "IsActiveMember" equals 1, groups the data by "Geography", and counts active customers in each region. The results are shown, and SparkSession is terminated.

Output:

```
[ec2-user@ip-172-31-41-39 ~]$ python3 actcust_reg.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/04 19:59:47 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+
|Geography|ActiveCustomers|
+-----+-----+
| Germany|          1248|
|  France|          2591|
|   Spain|          1312|
+-----+-----+
```

Task 3: Store Processed Data Back to S3

- **Objective:** Save the processed and aggregated data to a new S3 bucket or folder.
- **Steps:**
 1. Export data in CSV or Parquet format.
 2. Upload the processed data to a designated S3 location for easy access.

(Generating new CSV, downloading it and uploading to S3 bucket is fine).

```
GNU nano 8.3                                     upload.py
import boto3
import os

def upload_folder_content_as_file(local_folder, s3_bucket, s3_filename):
    s3_client = boto3.client('s3')

    # 1. Find the actual CSV file inside the folder
    try:
        files = os.listdir(local_folder)
        # Filter for files that end in .csv and ignore hidden/success files
        csv_files = [f for f in files if f.endswith('.csv') and not f.startswith('.') and not f.startswith('_')]

        if not csv_files:
            print(f"Error: No .csv files found inside {local_folder}")
            print(f"Contents found: {files}")
            return

        # We grab the first valid CSV found (usually part-00000...)
        actual_file_name = csv_files[0]
        local_file_path = os.path.join(local_folder, actual_file_name)

        print(f"Found actual file: {local_file_path}")
        print(f"Uploading to s3://{s3_bucket}/{s3_filename}...")

        # 2. Upload it
        with open(local_file_path, 'rb') as data:
            s3_client.upload_fileobj(data, s3_bucket, s3_filename)

        print("Success! Upload complete.")

    except Exception as e:
        print(f"Error processing folder: {e}")

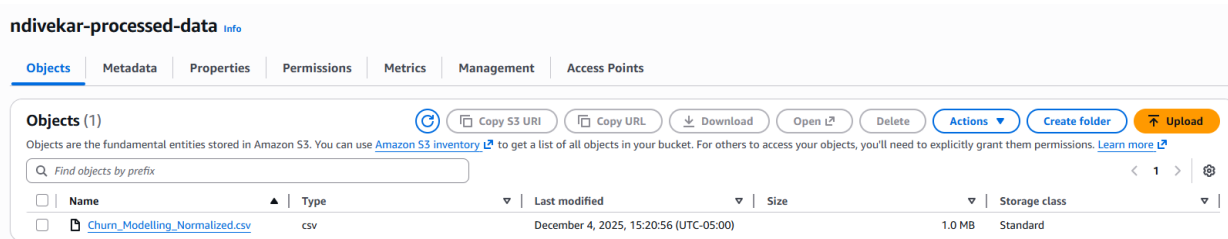
# --- PARAMETERS ---
folder_name = "Churn_Modelling_Normalized.csv" # The folder causing the error
bucket_target = "ndivekar-processed-data"
s3_key_name = "Churn_Modelling_Normalized.csv" # The clean name for S3

# --- EXECUTE ---
if os.path.isdir(folder_name):
    upload_folder_content_as_file(folder_name, bucket_target, s3_key_name)
else:
    print(f"Error: '{folder_name}' is not a directory. Please check your path.")
```

This Python script uploads all files in a given local directory to an Amazon S3

bucket. It uses Boto3 to setup an S3 client, then traverses the local directory and uploads each file to the bucket specified by the file name.

Output:



Task 4: Data Analysis Using Spark SQL

Objective: Use SQL to derive insights

Queries:

1. Identify regions with highest total account balance

```
# Query 1: Identify top-performing regions (by total Balance)
query1 = """
SELECT Geography, SUM(Balance) AS TotalBalance
FROM churn_data
GROUP BY Geography
ORDER BY TotalBalance DESC
"""

top_regions = spark.sql(query1)
print("Top-performing regions:")
top_regions.show()
```

Output:

```
Top-performing regions:
+-----+-----+
|Geography|      TotalBalance|
+-----+-----+
|   France|3.1133247949000007E8|
| Germany| 3.004028613799998E8|
|   Spain| 1.5312355201E8|
+-----+-----+
```

2. Analyze monthly revenue growth based on the Tenure Field, which acts as a proxy for months.

```
# Query 2: Analyze month-over-month revenue growth (based on Balance and Tenure)
query2 = """
SELECT Tenure AS Month, SUM(Balance) AS TotalBalance
FROM churn_data
GROUP BY Tenure
ORDER BY Month
"""

monthly_revenue_growth = spark.sql(query2)
print("Month-over-month revenue growth:")
monthly_revenue_growth.show()
```

Output:

```
Month-over-month revenue growth:
+-----+-----+
|Month|      TotalBalance|
+-----+-----+
| 0|3.2989990089999985E7|
| 1| 8.130482267000003E7|
| 2| 8.052792854000005E7|
| 3| 7.836707242000008E7|
| 4| 7.324474072999993E7|
| 5| 7.736161223000006E7|
| 6| 7.194861253E7|
| 7| 7.766708019000001E7|
| 8| 7.988501321000001E7|
| 9| 7.533754331000002E7|
|10|3.6224476960000016E7|
+-----+-----+
```

3. Determines the most popular product categories based on the count of customers in each category (NumofProducts).

```
# Query 3: Determine the most popular product categories (by NumOfProducts)
query3 = """
SELECT NumOfProducts, COUNT(*) AS CustomerCount
FROM churn_data
GROUP BY NumOfProducts
ORDER BY CustomerCount DESC
"""

popular_products = spark.sql(query3)
print("Most popular product categories:")
popular_products.show()
```

Output:

```
Most popular product categories:
+-----+-----+
|NumOfProducts|CustomerCount|
+-----+-----+
|          1|         5084|
|          2|         4590|
|          3|          266|
|          4|           60|
+-----+-----+
```


- Calculates the percentage of active customers (IsActiveMember) in each region.

```
# Query 5: Percentage of active customers by region
query5 = """
SELECT Geography,
       COUNT(*) AS TotalCustomers,
       SUM(CASE WHEN IsActiveMember = 1 THEN 1 ELSE 0 END) AS ActiveCustomers,
       (SUM(CASE WHEN IsActiveMember = 1 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) AS ActivePercentage
FROM churn_data
GROUP BY Geography
ORDER BY ActivePercentage DESC
"""

active_customers_percentage = spark.sql(query5)
print("Percentage of active customers by region:")
active_customers_percentage.show()
```

Output:

```
Average Estimated Salary by Gender:
+-----+-----+
|Gender|    AverageSalary|
+-----+-----+
|Female|100601.54138234648|
|  Male| 99664.57693054806|
+-----+-----+
```

- Computes the average salary for male and female customers.

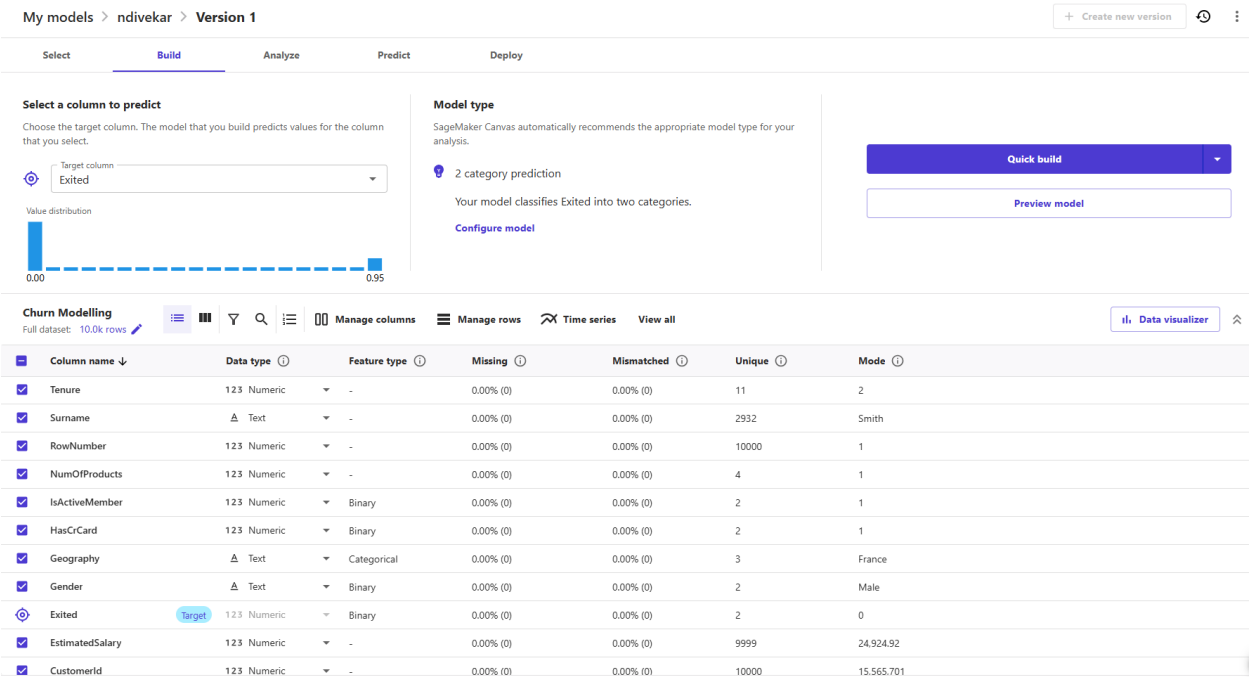
```
# Query 4: Average Estimated Salary by Gender
query4 = """
SELECT Gender, AVG(EstimatedSalary) AS AverageSalary
FROM churn_data
GROUP BY Gender
"""

average_salary_gender = spark.sql(query4)
print("Average Estimated Salary by Gender:")
average_salary_gender.show()
```

Output:

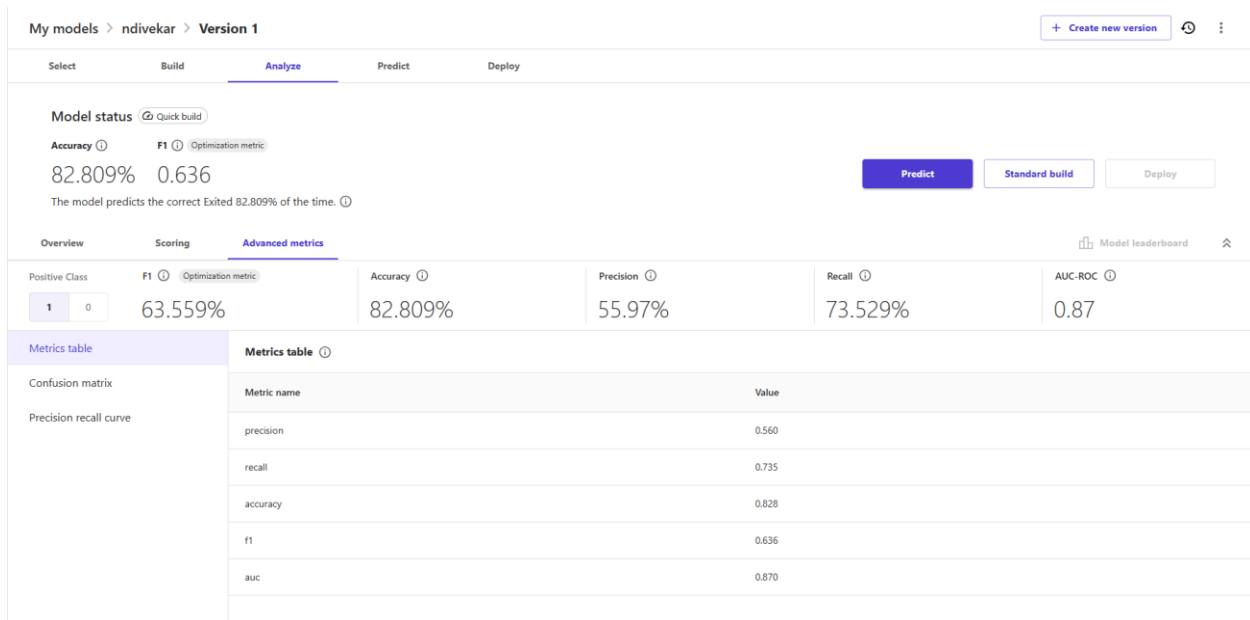
```
Percentage of active customers by region:
+-----+-----+-----+-----+
|Geography|TotalCustomers|ActiveCustomers| ActivePercentage|
+-----+-----+-----+-----+
|    Spain|         2477|          1312|52.96729915220024|
|   France|         5014|          2591|51.67530913442361|
|  Germany|         2509|          1248|49.74093264248705|
+-----+-----+-----+-----+
```

Task 5: Machine Learning with AWS SageMaker Autopilot



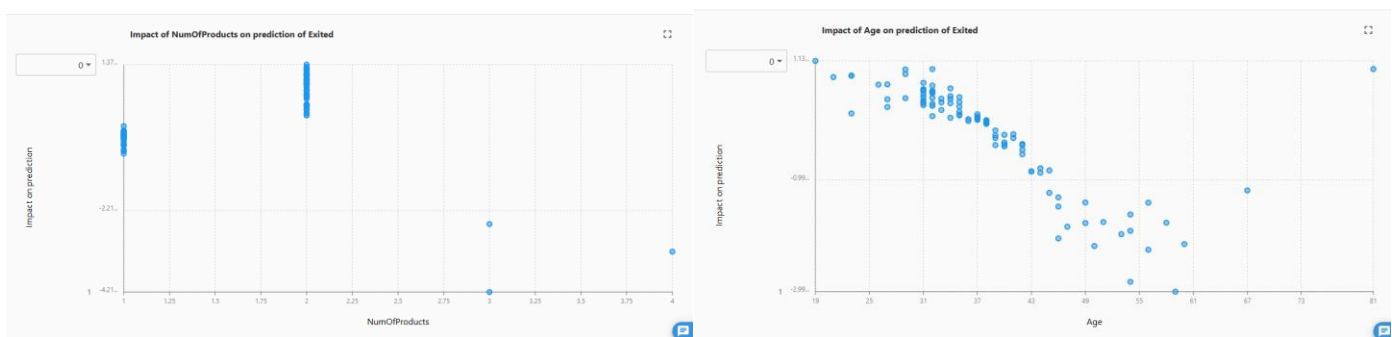
Let's break down what we're seeing in that Amazon SageMaker Canvas image for the churn model. It's a pretty straightforward setup:

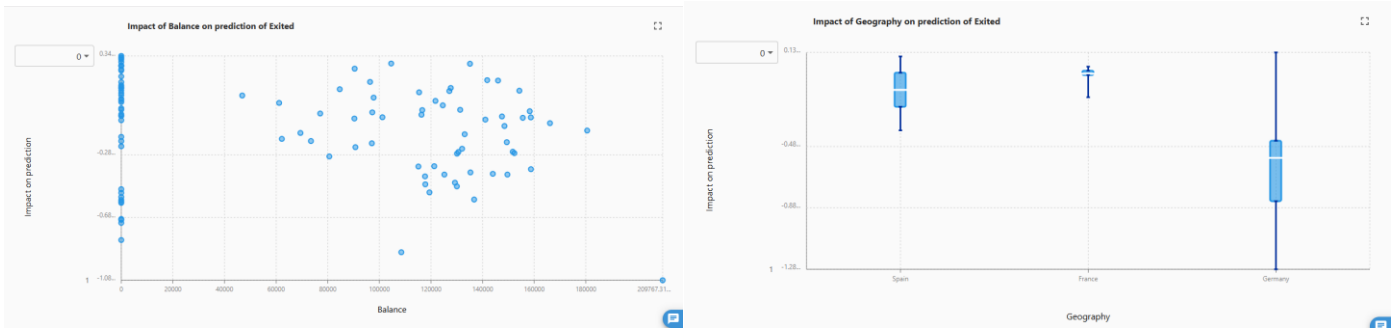
- The Goal (Target Column):** The primary objective is predicting customer turnover, or **churn**. That's why the “**Exited**” column is selected as the target. The model is being trained to figure out if a customer will leave or stick around.
- The Model Recommendation:** SageMaker is smart; it automatically suggested a “**2-category prediction.**” This just means it knows we're dealing with a simple *yes/no* question (binary classification): Will the customer exit, or won't they?
- The Data:** We're working with a solid dataset that has **10,000 customer records** and **14 data columns** (features). A great sign is that the interface is showing **0% missing values** across the board. We can also peek at other helpful stats like how many unique values and the most common value (the mode) for each column.



The analysis tab in Amazon SageMaker Canvas confirms the machine learning model has completed a Quick Build, yielding initial performance statistics. The overall Accuracy of the model registered at 82.809%, indicating that the predictive capability correctly classified the “Exited” target column approximately 82.8% of the time.

The optimization metric used for balancing classification quality, the F1 Score, achieved a value of 0.636. A deeper examination of the advanced metrics reveals a Precision of 55.97%, which measures the accuracy of true positive predictions, and a Recall of 73.529%, which measures the percentage of true positives that the model successfully captured. Furthermore, the Area Under the Curve (AUC-ROC) value of 0.87 indicates a strong capacity for overall classification performance.





The model's performance on the sample dataset is visually represented through a Sankey chart, which maps the predicted versus actual outcomes. The chart indicates that the majority of correct classifications are concentrated in Class 0 (non-exited) outcomes, depicted by the blue flows, while all incorrect predictions are distinctively represented by the pink flows. Out of 2,001 total predictions evaluated in this sample, 1,593 were correct classifications specifically for Class 0.

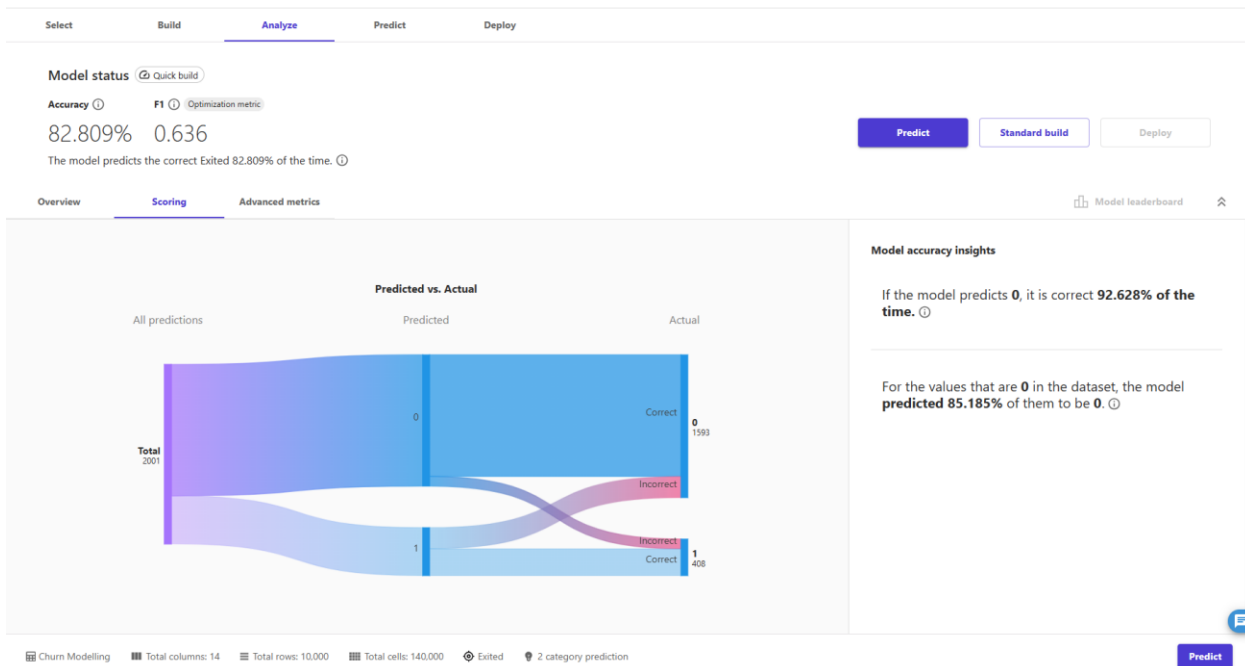
The corresponding Model Accuracy Insights provide specific performance metrics for this majority class. The model demonstrates high Precision for Class 0, being correct **92.628%** of the time when it forecasts a non-exited outcome. Furthermore, the Recall for Class 0 is **85.185%**, indicating that the model successfully identified **85.185%** of all actual Class 0 values present within the dataset.

A **Sankey chart** illustrates the flow of predicted versus actual outcomes:

- Most correct predictions belong to class 0 (non-exited), represented in blue.
- Incorrect predictions are highlighted in pink.
- Total predictions: 2,001, with 1,593 correct predictions for class 0.

Model Accuracy Insights:

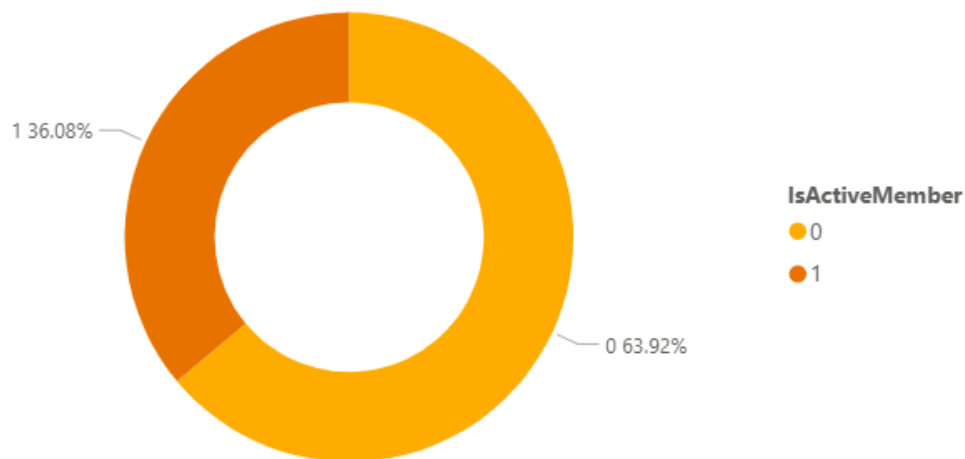
- When the model predicts class 0, it is correct 92.628% of the time.
- Among all actual class-0 cases, the model predicts class 0 with 85.185% accuracy.



4. Visualization

1. This visualization shows a pie chart titled “Sum of Exited by Isactivemember”. It visualizes the sum if customer exits grouped by their active membership status. The chart contains two segments:

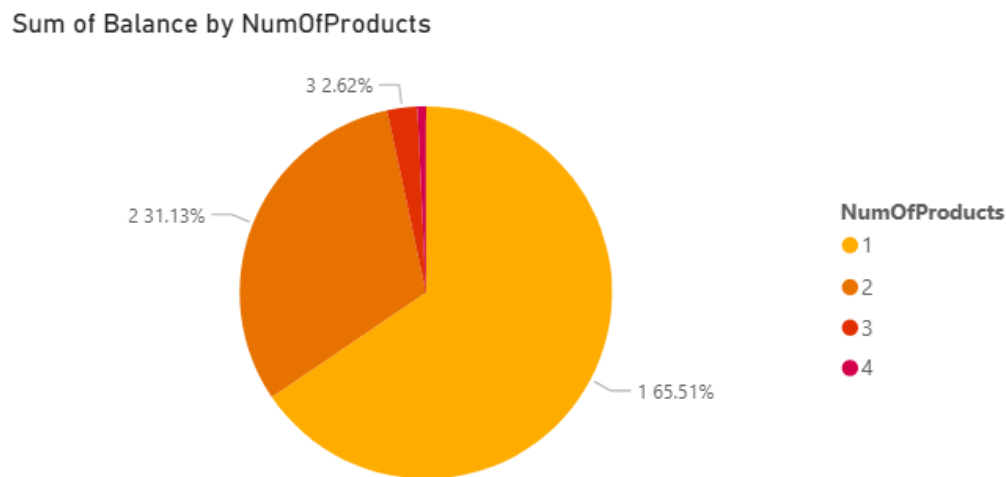
Sum of Exited by IsActiveMember



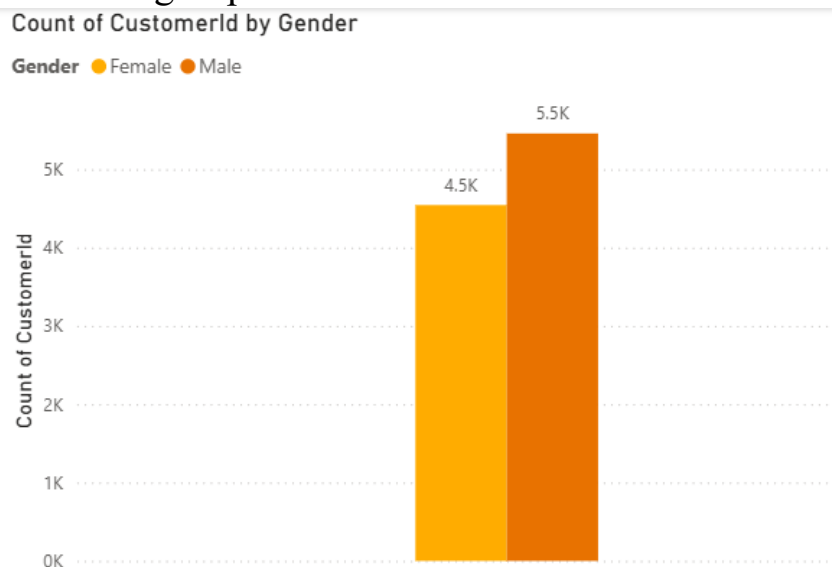
1: Represents by an orange section, it corresponds to active members.

0: Represents by a yellow section, it corresponds to inactive members.

2. This visualization displays a pie chart “Sum of balance by NumofProducts”. The chart represents the total balance, 764.86M, distributed across customers grouped by the number of products they use.

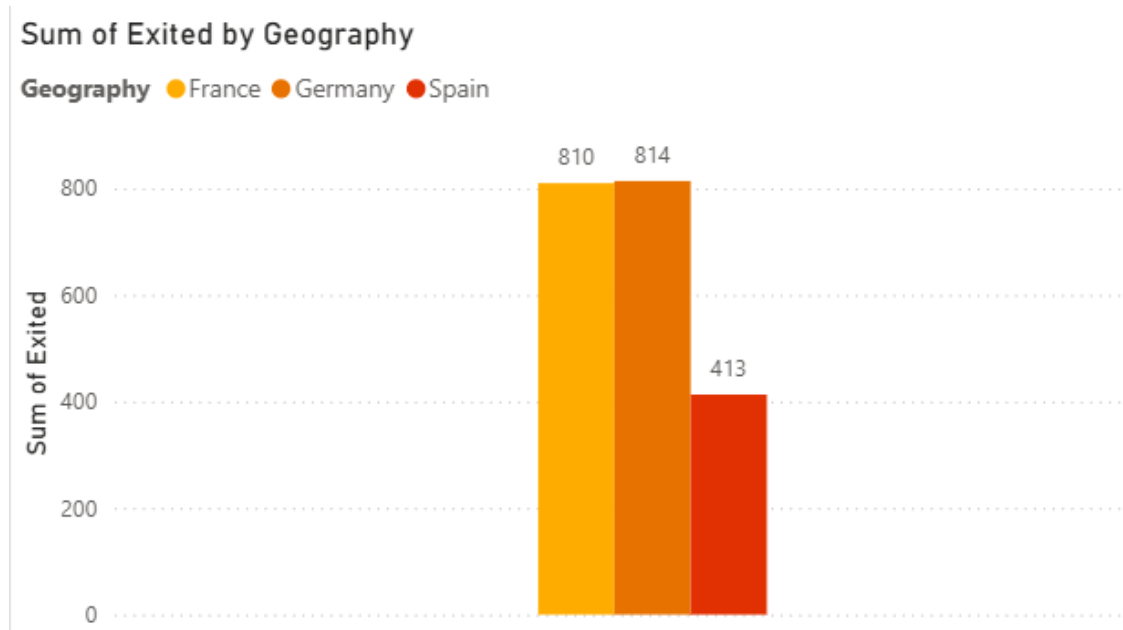


3. This visualization displays a bar chart titled “Count of records by Gender”. It visually represents the count of records categorized by gender into two groups: Male and Female.



- The Male category has a taller bar, indicating a count close to 5.5k records.
- The Female category has a shorter bar, showing a count slightly above 4.5K records.

4. This image shows a bar chart titled “Sum of Exited by Geography”. It visualizes the total number of customers exits across three geographic regions: Germany, France, and Spain.



- a. Germany and France have the highest count of exits, with their bars reaching close to 800.
- b. Spain has significantly fewer exits, with the bar slightly above 400.

Live link to the visualizations:

<https://app.powerbi.com/view?r=eyJrIjojNjNIYTl3NzMtYzk4Yy00ZWE4LWI3Y2YtNDMxMWEzNmViN2NjIiwidCI6IjExMTNiZTM0LWFlZDEtNGQwMC1hYjRiLWNkZDAyNTEwYmU5MSIsImMiOjN9>